Davante

Desarrollo de Aplicaciones Web
Desarrollo Web Entornos Cliente

Tema 5.
Los objetos de lenguaje

Diego Rodero Pulido

Tarea 5. Manipular el DOM

Tema 5. Los objetos de lenguaje · Tarea individual · Calificación: 5 puntos por el trabajo + hasta 5 puntos en la defensa en clase.

Objetivo

Dar el salto de la consola a la página: desde este tema, el JavaScript lee y cambia lo que se ve. Al terminar la tarea debes ser capaz de:

- Seleccionar elementos con `querySelector`, `querySelectorAll` y `getElementById`.
- Cambiar su contenido con `textContent` y su aspecto con `classList`, y explicar por qué `innerHTML` es peligroso con lo que escribe el usuario.
- Crear elementos con `createElement` y `append`, y eliminarlos con `remove()`.
- Consultar datos de la ventana y del navegador con el BOM: `window`, `screen`, `location` y `navigator`.

Qué vas a construir

En la carpeta `tema05` de tu repositorio de DWEC vas a completar una página, «Manipular el DOM», con seis ejercicios. Esta vez **el HTML te lo doy hecho**: es la página que vas a manipular, y tu trabajo está en `js/app.js`.

- Está en la carpeta `tema05_plantilla` del GitHub de la asignatura: https://github.com/DRodero/DWEC_2627. Cópiala en tu repositorio y cámbiale el nombre a `tema05`. Trae esto:

```
tema05_plantilla/
├── index.html     la página, ya maquetada con Bootstrap
├── js/
│   └── app.js     una función por botón, con modelos y TODO
├── capturas/      imágenes de relleno con el nombre de cada captura
└── README.md      el guion del README, con sus apartados
```

- Todo lo que va entre **[corchetes]** es un hueco: sustitúyelo por lo tuyo. Y cuando completes un `TODO`, borra el comentario.

Requisitos:

- Tu **nombre y apellidos** en la `navbar`, en el `<title>` y en el pie de la página.
- No cambies los `id` ni las clases que usa el JavaScript (`#contenido`, `#primero`, `.segundo`, `#nuevoTexto`...). Puedes mejorar el aspecto con Bootstrap, pero manteniéndolos.

- `app.js` está enlazado en el `<head>` con `defer`: no lo quites. Sin él, el script se ejecutaría antes de que exista el HTML y `querySelector` devolvería `null`.
- El texto se cambia con `textContent` y el aspecto, con `classList` y las clases de Bootstrap (`bg-warning-subtle`, `text-danger`, `d-none`...), no con `style`. Sin CSS propio.
- `let` y `const`, nunca `var`; comparaciones con `===`; comillas rectas; y cada función deja su rastro en la consola con `console.log()`.

Ejercicio 1. Selectores y modificación

El botón «Aplicar cambios» cambia los tres párrafos de `#contenido`, cada uno con un selector distinto:

1. El primero, con `getElementById("primero")`: su texto pasa a ser «Texto modificado por JavaScript». Viene hecho como modelo.
2. El segundo, con `querySelector` y su clase `.segundo`: se resalta con la clase `bg-warning-subtle`.
3. El tercero, con `querySelectorAll("#contenido p")` y su posición en la lista: se pone en rojo con la clase `text-danger`.

Ejercicio 2. Crear y eliminar nodos

1. «Añadir párrafo» crea un `<p>` con `createElement`, le pone el texto «Párrafo nuevo N» (N es su número dentro de `#contenido`) y lo cuelga al final con `append`.
2. «Eliminar último párrafo» borra el último párrafo de `#contenido` con `remove()`.
3. Si ya no queda ninguno, `lastElementChild` es `null`: en vez de un error en la consola, muestra el aviso `#aviso2` quitándole la clase `d-none`.

Ejercicio 3. Resaltar al pasar el ratón

1. «Activar resaltado» recorre los párrafos de `#contenido` con `for…of` y a cada uno le añade dos eventos: al entrar el ratón (`mouseenter`) se pone gris con la clase `bg-body-secondary`, y al salir (`mouseleave`) se quita.
2. `addEventListener` todavía no lo hemos visto: la plantilla trae hecho el de `mouseenter` como modelo, y tú escribes el de `mouseleave`. Los eventos, a fondo, en el tema 9.
3. Añade párrafos con el ejercicio 2 y comprueba qué pasa con los nuevos. Explica en el README por qué hay que volver a pulsar el botón.

Ejercicio 4. Cambiar el primer párrafo

1. «Cambiar texto» lee lo escrito en `#nuevoTexto` con `value` y `trim()`, y lo pone en el primer párrafo de `#contenido` con `textContent`.
2. Si la casilla está vacía, no cambia nada y muestra el error `#error4` quitándole `d-none`; si no, lo oculta.
3. «Probar con innerHTML» hace lo mismo con `innerHTML`. Escribe `<b>hola</b>` y prueba los dos botones: explica en el README qué diferencia ves y por qué nunca se mete en `innerHTML` lo que escribe el usuario.

Ejercicio 5. Lista de tareas

1. «Añadir» crea un `<li class="list-group-item">` con una casilla de verificación (`<input type="checkbox" class="form-check-input">`) y un `<span>` con el texto de `#tareaInput`, lo cuelga de `#listaTareas` y vacía la casilla. Si está vacía, avisa en `#error5`.
2. «Borrar completadas» selecciona las casillas marcadas con `querySelectorAll("#listaTareas input:checked")`, las recorre con `for…of` y borra el `<li>` de cada una, que es su `parentElement`. Si no hay ninguna marcada, avisa.
3. El contador `#totalTareas` muestra cuántas tareas hay en la lista después de añadir y de borrar.

Ejercicio 6. Datos de la ventana (BOM)

1. «Ver datos» rellena la tabla con el tamaño de la ventana (`innerWidth` × `innerHeight`, que viene hecho como modelo), el de la pantalla (`screen.width` × `screen.height`), la URL (`location.href`) y el idioma del navegador (`navigator.language`).
2. Con `if` y `else if`, el punto de corte de Bootstrap según el ancho de la ventana: xs (menos de 576 px), sm (desde 576), md (768), lg (992), xl (1200) y xxl (1400).
3. Pruébalo en tu ordenador y con el modo dispositivo de F12, como si fuera un móvil.

Evidencias y reflexión (README.md)

En el `README.md` de la carpeta explicas lo que has hecho y lo demuestras; la plantilla trae el guion con los apartados. Las capturas van en la carpeta `capturas/`, tienen que estar hechas en tu equipo y cada una lleva debajo una o dos líneas que expliquen qué se ve.

1. Seis capturas propias, que sustituyen a las imágenes de relleno de `capturas/` con el mismo nombre: la página recién cargada, con tu nombre visible; los ejercicios 1 y 2 aplicados; un párrafo resaltado con el ratón encima; `<b>hola</b>` puesto con innerHTML; la lista con tres tareas y una marcada; y los datos de la ventana en modo móvil.
2. Una reflexión de 5 a 8 líneas: ¿qué diferencia hay entre `textContent` e `innerHTML`, y por qué no se mete en `innerHTML` lo que escribe el usuario? Pon como ejemplo lo que pasó con `<b>hola</b>`.
3. Fuentes consultadas, con su enlace, y el apartado «Uso de IA» si corresponde.

Entrega

- Todo va en la carpeta `tema05` de tu repositorio público de DWEC, con esta estructura:

```
tema05/
├── index.html     la página de la plantilla, con tu nombre
├── js/
│   └── app.js     una función por botón
├── capturas/      las seis capturas
└── README.md      capturas, reflexión y fuentes
```

- Sube el trabajo en varios commits a medida que avances (tres como mínimo), con mensajes que digan qué cambia: «Ejercicio 1: selectores», «Lista de tareas»...
- En la tarea del campus virtual entregas solo la URL de la carpeta: `https://github.com/tu-usuario/tu-repositorio/tree/main/tema05`
- Extensión orientativa del README: de 200 a 400 palabras, más las capturas.
- Se corrige el último commit anterior a la fecha límite publicada en la propia tarea del campus; lo que subas después no cuenta. No se admiten entregas por otros medios.

Calificación

El trabajo vale 5 puntos, repartidos por ejercicio. Los otros 5 puntos, hasta el 10, se obtienen en la defensa en clase.

Apartado	Puntos	Qué se valora
Ejercicio 1. Selectores y modificación	0,5	Los tres párrafos cambian, cada uno con un selector distinto, y con clases, no con style.
Ejercicio 2. Crear y eliminar nodos	0,75	Añade párrafos numerados con createElement y append, borra el último con remove() y avisa cuando no queda ninguno.
Ejercicio 3. Resaltar al pasar el ratón	0,5	mouseenter y mouseleave en todos los párrafos, y explica en el README qué pasa con los párrafos nuevos.
Ejercicio 4. Cambiar el primer párrafo	0,75	value y trim, error si la casilla está vacía, y la prueba de textContent frente a innerHTML explicada.
Ejercicio 5. Lista de tareas	1	Añade tareas con su casilla y vacía el input, borra las marcadas, avisa si no hay y mantiene el contador.
Ejercicio 6. Datos de la ventana	0,5	Los cinco datos y el punto de corte correcto con if y else if.
Calidad del trabajo entregado	1	Código limpio, comentado y sin restos de la plantilla (huecos o TODO sin quitar); README cuidado, con las seis capturas propias comentadas y la reflexión; varios commits que muestran el avance.
Total	5	

Cada apartado se puntúa de 0 a su máximo, según lo que funcione, con cuartos de punto.

Defensa en clase (hasta +5)

La defensa vale **la mitad de la nota**: es donde demuestras que sabes hacer lo que has entregado. No se trata de repetirlo: te pediré uno o dos cambios pequeños, los harás en el momento explicando lo que haces y te preguntaré por tu código.

- **Cuándo:** en la sesión de clase que indique después del cierre del plazo. Unos cinco minutos por persona.
- **Requisito:** solo puedes defender la tarea si la has entregado en plazo. Si no la defiendes, tu nota es la del trabajo (máximo 5).
- **Cómo:** abres tu página desde la versión entregada, la enseñas funcionando con la consola abierta, haces en tu propio código el cambio que te pida y respondes a lo que te pregunte sobre él.

- **Sin IA:** durante la defensa desactiva Copilot o cualquier autocompletado con IA. Puedes consultar los apuntes y la documentación de MDN.

Ejemplos de lo que te puedo pedir

- Cambiar el selector de un párrafo, por ejemplo `getElementById` por `querySelector`, sin que cambie el resultado.
- Añadir un botón que borre el primer párrafo en lugar del último.
- Cambiar el color del resaltado o del error usando otra clase de Bootstrap.
- Quitar el `defer` del `<script>` y explicar el error que aparece.
- Explicar qué pasaría en la lista de tareas si usaras `innerHTML` con lo que escribe el usuario.
- Encontrar y corregir, con ayuda de la consola, un error que yo introduzca en tu código.

Cómo se puntúa

Puntos	Qué demuestras
5	Haces el cambio sin ayuda y con soltura, explicas con seguridad tu código y lo que aparece en la consola, y respondes bien a lo que te pregunto.
4	Haces el cambio sin ayuda, con alguna duda menor, y explicas bien lo esencial.
3	Haces el cambio con una pista pequeña y explicas correctamente la mayor parte de tu código.
2	Necesitas varias pistas para hacer el cambio o tu explicación tiene lagunas importantes.
1	No consigues hacer el cambio sin mucha ayuda y solo sabes explicar partes sueltas de tu código.
0	No consigues hacer el cambio ni explicar tu propio código, o no te presentas.

Se pueden dar valores intermedios de medio punto.

Nota final = trabajo (máximo 5) + defensa en clase (máximo 5).

Uso de la inteligencia artificial

Puedes apoyarte en herramientas de IA para orientarte, entender un error o revisar la redacción, pero el código y las explicaciones tienen que ser tuyos. Si las utilizas, indícalo al final del README en un apartado «Uso de IA»: qué herramienta, para qué y qué hiciste después con su respuesta. Declararlo no resta nota.

Advertencia: si se detecta que la tarea se ha generado con IA sin trabajo propio, la calificación será un 0 absoluto.

Son indicios: la falta de evidencias propias, el código con técnicas que no hemos visto y que no sabes explicar, los textos genéricos que no responden a lo que se pide, subir todo el trabajo de golpe en un único commit y no saber explicar lo entregado. Una captura tomada de internet, prestada o generada cuenta como el mismo caso.

La defensa en clase sirve precisamente para comprobarlo: si en ella no eres capaz de explicar nada de lo que has entregado, no solo no sumas esos 5 puntos, sino que se considera un indicio claro de que la tarea no es tuya.

Ejemplo de resultado esperado

